Copilot workflow optimisation: decided plan

Decided 2026-10-09 (grill-me interview, host `t16`). **P1 is built and accepted (2026-10-09, §6.12). P3 is built, installed in shadow mode and verified live (2026-10-10, §6.13); it is enabled only after 3 days of shadow log, your decision. P2 onward is not started.** Ollama offload (D15, §6.11) was added the same day by a second grill-me. In a fresh session: read this file, do not redo the analysis, tick §12 as you go. Start with: `cd ~/copilot && copilot` then "Read PLAN.md, confirm P0a, then do P2" (P1 and P3 are done; §6.12 and §6.13 list what they left open for you; run `ctx-hook review` first, §6.13 says how to read it).

1. Goal

- Cut AIU spend ~50% at the same task quality. Expected with the chosen levers: **−30–40%** (§10). Whether the gap closes is decided later from the weekly report (§11).
- Metric: AIU = `total_nano_aiu / 1e9` in `~/.copilot/session-store.db` → `assistant_usage_events`.
- Quality guardrail: automated proxies only (§6.8).

2. Evidence (baseline 2026-06-08 → 2026-10-09: 324 sessions, 188 with usage, ~18.7k calls, ~129.4k AIU)

- AIU by token type (exact, from `token_details_json`): cache_read 53.0%, cache_write 29.9%, output 14.7%, uncached input 2.3%. Context carrying is 83%; `effort: max` is a minor lever.
- Sonnet-5 price table (AIU per 1M tokens): input 200, cache_read 20, cache_write 250, output 1000. Prompt-cache TTL is 300 s, so a cold rebuild costs ~12× a warm read.
- 81.5% of AIU comes from calls with ≥100k-token prompts, 21.6% from >400k. 199 of the 200 sessions that record a tier run `long_context` (sonnet-5 prompt window 936k).
- Concentration: 12 sessions = 50% of AIU; 30 sessions >1,000 AIU = 72%; 15 sessions >2,000 AIU = 55%.
- Autopilot: 86 of 188 sessions = 71.8% of AIU (1,089 AIU and 153 calls per session, vs 350 and 56).
- Cold-cache rebuilds (cache_read <50% of prompt, gap ≥5 min): 461 calls = 19.9% of AIU.
  - User-initiated: 247 calls = 12.4% (avg idle 193 min, avg context 231k, ~65 AIU each). A fresh session start costs ~11 AIU.
  - Agent-initiated: ~187 calls = 7.3%. Long spans in the gap: `ask_user` 52, bash calls ≥4 min ~55 (mostly approval waits), `sleep`/`read_bash` polls 27, sub-agents 13.
- Carry-cost model (item size × later calls that re-read it; estimates): fixed baseline ~15% (≈42k tokens at call 1: system ≈13k, tool schemas ≈13k for 29 tools ≈450 each, instructions, 44 skills ≈2.5k), assistant messages + tool args ~12%, `view` ~9%, bash search/read ~3%, builds ~3%, web ~1.5%.
- No main-context tool result exceeds 8k tokens (the CLI caps them). Share of AIU by result size: <1k 8.2%, 1–3k 6.4%, 3–8k 2.8%.
- Parallelism: only 13.8% of tool-calling messages issue more than one call; at most 9.5% of calls could merge.
- Sub-agents ≈11% of AIU; `gpt-5.4` alone 7% (2,219 calls, 4.1 AIU/call, avg context 86k); `gpt-5.4-mini` 0.8 AIU/call.
- Permissions: 6,607 prompts, 86 h cumulative wait, 540 waited ≥60 s, 129 ≥300 s. Top triggers: compound `cd … &&` shell 817, grep 482, echo 239, git 236, cat 220, find 211, ls 167; reads in `~/kms.split_key/crate` 196, `~/kms/crate` 191, `~/pr-998/crate` 117.
- Quality-proxy baselines: `abort` 74 per 2,596 user messages (≈2.9/100); `model.model_call_failure` + `model.turn_retry` + `session.error` = 353 per 24.7k assistant turns (≈1.4/100); `session.task_complete` 272; frustration classifier fired 2 of 477 (unusable); user-rejected tool calls 13.

- Secrets in logs (counts only, values never read): password/secret assignments in 35 of 350 sessions, PEM private keys 5, AWS-style keys 5, JWT 6, bearer 2.
- `events.jsonl`: 350 files, 663 MB, dominated by base64 `session.binary_asset` (12 of 21 MB in one session).

3. Decisions

#	Decision	Why
D1	Success = −50% AIU at same quality; expect −30–40%	§2, §10
D2	Default context tier everywhere; `long_context` only via `--context long_context`	83% of AIU is context carrying; default compacts at ~160k, long_context at ~750k
D3	Short sessions + recall: PG log, handoff note auto-injected by a `sessionStart` hook, raw search as drill-down	20% of AIU is cold rebuilds; fresh start ≈11 AIU vs ≈65 for a stale resume
D4	Handoff writer is hybrid: deterministic extractor refreshed every turn end, upgraded by any checkpoint or model-written handoff. The 324 old sessions get deterministic digests now; an Ollama (`gemma4`) backfill runs only if the §6.11 bake-off passes	zero AIU, needs no discipline; a 3–8B model can invent decisions
D5	Redact at ingest (single choke point); raw text stays only in the CLI's own `events.jsonl`	secrets found in 35 sessions
D6	Rust Cargo workspace in `~/copilot` (sqlx, rmcp, tree-sitter)	user choice; millisecond hooks
D7	Debian-native PostgreSQL 17 + pgvector, unix-socket peer auth	distro-signed, no password to store
D8	`permissionRequest` hook: tight read-only allow-list plus reads in sibling worktrees; shadow mode first	6,607 prompts, 129 waited ≥5 min. As built and measured (§6.13): reads never reach the hook, and the allow-list covers 0 of the 116 shell-request waits ≥5 min
D9	MCP: 3 tools (`history_search`, `history_get`, `code_search`). Code index lexical + symbols first; embeddings only if replay evaluation says so	~450 tokens/tool/call
D10	Explore/task sub-agents on a cheaper model via `/subagents`	sub-agents 11% of AIU
D11	Per-session soft cap `maxAiCredits` = 1,000	30 sessions >1,000 AIU = 72% of spend
D12	Nudges: batch independent calls, poll ≤240 s, delegate noisy loops, autopilot alias with cap and fresh start	§6.10
D13	Quality = weekly automated report vs baseline; flag a proxy >25% worse for 2 consecutive weeks	
D14	Rollout: today default tier + soft cap; P1 and P3 in parallel; P0b after a week of data	attribution
D15	Ollama is a background helper, not a Copilot substitute: Route (a minimal `lq` shell function, explicit use) + Background (P2 digests, P4 search), each behind a measured gate (§6.11). Model `gemma4` (`think:false`), fallback `llama3.2:3b`. No Delegate tool, no BYOK/Swap. Batch work shares the main `ollama.service`; CPU contention with builds is accepted	1-call turns are 2.7% of AIU (≈30 AIU/week of ≈13k); CPU-only host (`gemma4`: 22 tok/s prompt, 6.5 tok/s generation)

4. Dropped (do not revisit without new data)

- `postToolUse` result-trimming hook: no result >8k tokens, ≤2% gain, quality risk.

- Web/doc cache or mirror: web fetch/search is ≈1.5% of AIU.
- LLM gateway / network proxy: needs TLS interception or BYOK.
- Ollama as a Copilot tool (Delegate) or as the model for sessions and sub-agents (Swap, BYOK `COPILOT_PROVIDER_*`): Copilot's ≈42k-token fixed prompt takes ≈30 min to ingest at ≈22 tok/s on this CPU, a tool schema costs ≈450 tokens per call, and 1-call turns are only 2.7% of AIU.
- Registering `graphify-mcp`: 10 tools ≈ +3–4k tokens per call, one graph per server. Keep graphify as skill/CLI.
- Effort `max` → `high` and disabling unused skills: parked in §11.

5. Phases

Order: P0a → (P1 ‖ P3) → P2 → P0b (after ≥1 week of data under P0a; use P1's report, or direct SQLite queries if P1 is late) → P4 → P5.

Phase	Work	Acceptance
P0a (today, no code)	`~/.copilot/settings.json`: `"contextTier": "default"`. Soft cap: check whether `/limits` persists, else alias `copilot --max-ai-credits 1000`. Alias `copilot-long='copilot --context long_context'`.	New session's `session.start` event shows `contextTier: default`; limit visible in `/limits`.
P1	Cargo workspace, DB, ingester (backfill + tail), redaction, report views (§6.2–6.4, 6.8).	Row counts match sources; re-run adds no duplicates; report reproduces §2 within 1% (≈129.4k AIU, token-type split); zero secret-pattern matches in the DB (assert on counts, never print values).
P3	`ctx-hook permission` in shadow mode, then enabled (§6.6).	p95 <50 ms; offline replay of the historical `permission.requested` events (6,607 in §2; P1 counts 7,195 events / 6,907 distinct `requestId`s, §6.12) gives 0 false allows on write, delete, network or privilege cases; 3 days of shadow log reviewed before enabling. **Status (§6.13): built and installed in shadow mode; p95 and replay met; the 3 days have not started.**
P2	Deterministic note builder + `ctx-hook session-start` (§6.5); deterministic digests for all old sessions; then the §6.11 bake-off, and an Ollama (`gemma4`) backfill only if it passes.	Note ≤1.5k tokens, hook p95 <100 ms, injected on `startup`/`new` only; verified through `hook.start` events.
P0b	`/subagents`: explore + task → cheaper model (try `gpt-5.4-mini`, compare `claude-haiku-4.5`; leave rubber-duck, code-review, security-review, research alone). Nudges in `~/.copilot/copilot-instructions.md` (§6.10). Alias `copilot-auto='command copilot --autopilot --max-ai-credits 600'` (`command` skips the P0a `copilot` cap alias so the flag is not doubled; per objective: `/autopilot <goal> --max-ai-credits N`).	Calls/turn and sub-agent AIU/call fall; no proxy >25% worse.
P4	`ctx-mcp` with `history_search`, `history_get`; Postgres full-text search only, and `nomic-embed-text` is pulled into Ollama only if the §6.11 replay-eval says so.	All-tools schema ≤1.5k tokens (`tool_tokens` in `session.usage_checkpoint`), responses ≤800 tokens, p95 <300 ms.
P5 (gated)	`code_search` (§6.9).	Replay-eval report, then the embeddings decision; measured drop in exploration calls per turn.
P6 (2-week trial)	`lq` shell function (§6.11), independent of P1–P5.	≥20 calls in 14 days and ≥70% of rated answers 'useful'; otherwise delete it.

6. Design notes

6.1 PostgreSQL (you run these: sudo needs a password; `apt-cache` showed no candidate for postgresql-17, so `apt update` first)

```
sudo apt update && sudo apt install postgresql-17 postgresql-17-pgvector
sudo -u postgres createuser cscm && sudo -u postgres createdb -O cscm copilot_ctx
sudo -u postgres psql -d copilot_ctx -c 'CREATE EXTENSION vector; CREATE EXTENSION pg_trgm;'
# socket only: set listen_addresses = '' in /etc/postgresql/17/main/postgresql.conf, then
sudo systemctl restart postgresql
psql -d copilot_ctx -c 'select 1'      # peer auth, no password
```

`DATABASE_URL=postgres:///copilot_ctx?host=/var/run/postgresql`

State on `t16` (2026-10-09): PostgreSQL 17.11 runs; role `cscm` (no superuser, no CREATEDB), DB `copilot_ctx`, extensions `plpgsql` and `pg_trgm`; the `select 1` above works. **Still open (you, sudo), and P1 needs neither:**

- `postgresql-17-pgvector` is not installed (`pg_available_extensions` lists no `vector`). P4's `hist_chunks.emb` needs it: `sudo apt install postgresql-17-pgvector`, then `sudo -u postgres psql -d copilot_ctx -c 'CREATE EXTENSION vector'`.
- `listen_addresses` is still `127.0.0.1`, not `''`. Not an open door (a TCP login without a password fails with `fe_sendauth: no password supplied`), but the "socket-only" mitigation of §9 is not met until you set it and restart PostgreSQL.

6.2 Schema sketch (names indicative)

- `sessions(id, cwd, git_root, repository, branch, model, effort, context_tier, autopilot, started_at, ended_at, name)`
- `turns(session_id, turn_index, user_message, assistant_final, ts)`
- `events(session_id, event_id, parent_id, ts, type, ctx, body jsonb)`; `ctx` is `main` or the sub-agent's `parentToolCallId`
- `tool_calls(session_id, tool_call_id, ctx, tool, args jsonb, result_text, ok, t_start, t_end)`; results capped at 8 KB; permission wait derived from `permission.*` events
- `usage(session_id, src_id, ts, turn_index, agent_id, model, initiator, effort, input, cache_read, cache_write, output, reasoning, nano_aiu, price jsonb)`
- `checkpoints`, `session_files`, `session_refs`: mirrored from SQLite
- `notes(id, git_root, branch, session_id, kind, body, tokens, updated_at)`; kind is deterministic, checkpoint, handoff or digest; checkpoint/handoff outrank deterministic
- `hist_chunks(id, session_id, turn_index, kind, text, tsv, emb vector(768) NULL)`
- P5: `code_chunks(repo_key, blob_sha, path, lang, symbol, kind, start_line, end_line, text, tsv)`, `worktree_files(worktree, path, blob_sha)`
- `ingest_state(source, key, offset, mtime)`, `redactions(session_id, kind, n)`, `hook_audit(ts, session_id, tool, cmd_sha, decision, rule, shadow)`
- As built for P1 (deviations, extra columns, the views): §6.12. `hook_audit` as built for P3 (14 columns, no foreign key to `sessions`): §6.13. `notes`, `hist_chunks` and the P5 tables do not exist yet.

6.3 Ingester (two sources are required)

- `~/.copilot/session-state/*/events.jsonl`: content (messages, tool calls/results, permissions, hooks, compaction). One JSON object per line with `type`, `data`, `id`, `parentId`, `timestamp`. Sub-agent events carry `data.parentToolCallId`. Skip `session.binary_asset.data`; keep assetId, mime and size.
- `~/.copilot/session-store.db` (open read-only, it is WAL): `assistant_usage_events` is the ONLY per-call source of cache_read/cache_write tokens and `token_details_json` (price table). `events.jsonl` has only aggregates (`session.usage_checkpoint`, `session.shutdown`). Also mirror `turns`, `checkpoints`, `session_files`, `session_refs`.

- Idempotent upserts keyed by event `id` / usage row id. Backfill first (350 files, 663 MB), then tail with `notify` plus a periodic rescan. Run at `nice`.

6.4 Redaction (before anything is stored, embedded or indexed)

Patterns: PEM private-key blocks; GitHub tokens (`ghp_`, `gho_`, `ghu_`, `ghs_`, `ghr_`, `github_pat_`); AWS `AKIA…`; JWT (`eyJ….….…`); `Bearer` + 30+ chars; `password`/`passwd`/`secret`/`api_key`/`token` assignments with a 12+ char value; Slack `xox[baprs]-`; Stripe `sk_live_`; plus an entropy rule for long random-looking tokens (tune thresholds). Replace with `[REDACTED:<kind>:<sha8>]`. Never print matches in logs or tests; assert on counts.

6.5 Handoff note (deterministic, ≤1.5k tokens)

- Key: (`git_root`, `branch`). Sections: goal (first user message, ≤400 chars), last 3 user asks (≤300 chars each), state (last assistant final message, ≤1,200 chars), files changed (≤20, from `session_files`), refs (PR/commit), open todos (from the session's `session.db` if present), latest checkpoint overview.
- A newer checkpoint or handoff outranks the extractor. Header: "Previous session on @, auto-generated, may be unrelated."
- Inject only when `source` is `startup` or `new` (never `resume`) and the note is ≤14 days old; if `initialPrompt` is present, prefer the best FTS match among the last 3 notes.
- Output `{"additionalContext": "..."}`. On any error print nothing and exit 0.

6.6 Permission hook

- **As built (P3): §6.13.** Three things differ from this design: reads never reach the hook (the CLI decides `read` requests before hooks run), the hook also fires for shell requests the CLI would approve by itself, and `cd` does not persist between shell calls. The measured result is that this allow-list answers 9.4% of the shell requests but almost none of the waiting.
- Config: `~/.config/ctx/permissions.toml`. Parse shell with tree-sitter-bash. Allow only if all hold:
  - every simple command is on the allow-list: `cd` into trusted roots only; `grep rg cat ls find head tail wc sort uniq jq echo printf which stat file du readlink`; read-only `git` (`log diff show status branch rev-parse ls-files blame describe`);
  - no redirection except `2>&1` and `/dev/null`; no `$(…)`, backticks or process substitution;
  - no `find -exec/-execdir/-ok/-delete/-fprint*`, no `sed -i/w`, no `xargs`, `awk`, `perl`, `python`.
- `read` requests (**not possible: they never reach the hook, §6.13; the roots and the deny list below judge the paths of shell commands instead**): allow under trusted roots (`~/kms*`, `~/pr-*`, extend in config) minus a deny-list (`**/.env*`, `*.pem`, `*.key`, `~/.ssh/**`, `~/.gnupg/**`, `~/.aws/**`, `~/.copilot/config.json`).
- Everything else: empty stdout = normal prompt. **Never exit 2 (it means deny).** Wrap `main` so any error exits 0 with empty stdout.
- Shadow mode first: log would-be decisions to `hook_audit(shadow=true)`, emit nothing. Step 1 is to log raw payloads: the input schema (`toolName`, `toolInput` incl. `requestSandboxBypass`) was not fully captured from the docs. Done: captured on a live 1.0.95 run (§6.13).
- Sample `~/.copilot/hooks/ctx.json`:

```
{
  "version": 1,
  "hooks": {
    "sessionStart": [{"type": "command", "exec": "/home/cscm/.cargo/bin/ctx-hook", "args": ["session-
start"], "timeoutSec": 5}],
    "permissionRequest": [{"type": "command", "exec": "/home/cscm/.cargo/bin/ctx-hook", "args": ["perm
ission"], "timeoutSec": 5}]
  }
}
```

6.7 MCP server

`rmcp` stdio. Tools: `history_search(query, repo?, since?, limit≤8)`, `history_get(ref)`, `code_search(query, repo?, lang?, limit≤8)` (P5). Descriptions ≤60 tokens each, results ≤800 tokens. Register with `/mcp` (no MCP config exists today; check where it writes). Do not register graphify-mcp.

6.8 Report and guardrail

- Views: weekly AIU; AIU per turn and per session; calls per turn; mean prompt tokens per call; AIU by token type and by initiator; autopilot share; sessions >1,000 AIU; cold-call share.
- Cold call = `input_tokens > 20000 AND cache_read_tokens*2 < input_tokens AND gap ≥ 5 min` since the previous call of the same (session, agent_id). AIU by token type = Σ `tokenCount*costPerBatch/batchSize` over `token_details_json`.
- Quality proxies (baselines in §2): aborts per 100 user messages; failures+retries+errors per 100 assistant turns; `session.task_complete` rate; correction-phrase rate (regex list TBD: "no", "wrong", "revert", "undo", ...); permission waits ≥5 min per week.
- Flag a lever when any proxy is >25% worse than the Jun–Oct baseline for 2 consecutive weeks. Use per-turn ratios: volume is uneven (Jul–Aug were quiet).
- As built: `ctx-report` (§6.12). The correction-phrase proxy is NOT implemented (the regex list is still TBD); every other proxy is.

6.9 Code index (P5, gated)

- Chunk with tree-sitter (Rust, Python, Bash, Nix, Lua, JS/TS, YAML, Markdown, Puppet). Postgres FTS with identifier splitting (snake_case, camelCase) + `pg_trgm`.
- Key chunks by git blob SHA so worktrees (`kms`, `kms.alpine`, `kms.split_key`, `pr-998`, ...) dedupe. Respect `.gitignore`, a deny-list and vendored/bundled excludes (graphify's `kms.hsm…` graph was polluted by Mermaid/Swagger bundles).
- Replay-eval: for past turns, query = user prompt or the agent's search pattern; relevant = files later viewed or edited in that turn. Report recall@5 and MRR. Add Ollama embeddings (`qwen3-embedding:0.6b` or `nomic-embed-text`) only if recall is poor (set the threshold after the first run).

6.10 Nudges (`~/.copilot/copilot-instructions.md`, P0b, ≈90 tokens)

```
- Batch independent reads/searches (view, grep, glob, read-only git) into ONE message as parallel
calls.
- While waiting on builds or CI, poll with read_bash delay <=240 s; never sleep >240 s (prompt cache
TTL is 300 s).
- Run long or noisy build/test/CI loops in a `task` sub-agent; return only a short summary and failing
excerpts.
```

6.11 Ollama offload (D15; `lq` trial = P6; gates for P2 and P4)

Decided 2026-10-09 (second grill-me). Ollama is a background helper, not a substitute for Copilot.

Why the gain is small. Main-agent 1-call turns are the only kind a local model could answer: 274 in the baseline (≈30/week), 3.2k AIU = 2.7% of spend (median ≈3.8 AIU in a 30-turn sample; the mean of 11.8 AIU is inflated by cold-cache rebuilds, which only D3 fixes). The other ≈94% are multi-call agentic loops: context carrying is 83% of AIU, and Copilot's ≈42k-token fixed prompt alone takes ≈30 min to ingest on this CPU. Realistic Route saving: <1% (≈30 AIU/week of ≈13k).

Measured on `t16` (Ollama 0.31.2, CPU-only: `size_vram` 0, 12 threads, ≈2k-token prompt):

Model	Prompt ingest	Generation	Cold load	RAM
`llama3.2:3b`	29 tok/s	6.8 tok/s	4.3 s	2.6 GB
`gemma4` (8B, `think:false`)	22 tok/s	6.5 tok/s	12.9 s	9.5 GB
`qwen2.5-coder` (7.6B)	15 tok/s	3.6 tok/s	6.5 s	5.1 GB

Speed only; quality is decided by the gates below. A typical `lq` call (500 tokens in, 200 out) takes ≈50 s warm on `gemma4`. Idle models unload after 5 min.

`lq` (P6). A ~20-line shell function (for example in `~/.zshrc` next to the P0a aliases), used explicitly by you. Copilot is not told about it: no tool, no nudge.

- `/api/generate` with `gemma4`, `think:false`, an explicit `num_ctx` (4096 covers the input cap) and streamed output; input from stdin or arguments.
- Refuses input over ~6k chars (≈70 s ingest). 180 s hang guard: a local call that blocks past the 300 s cache TTL cools Copilot's prompt cache (≈60 AIU to rebuild).
- Prints `[gemma4 local, unverified]` on stderr, then asks for a one-key rating (`y`, `n`, Enter = skip).
- Log: metadata only (timestamp, model, chars in/out, seconds, exit code, rating), never the prompt or the answer (secrets were found in 35 of 350 sessions, D5).
- Keep or kill after 14 days: ≥20 calls and ≥70% of rated answers 'useful' (skips do not count), otherwise delete the function.

P2 bake-off (gate for an Ollama backfill). Candidate `gemma4` (`think:false`); `llama3.2:3b` is tried only if `gemma4` fails.

- Test set: the 21 checkpointed sessions that have stored turns (23 checkpointed; excluded: `b42d5b81`, the Ollama interview itself, 0 stored turns, and `0a9e0714`, 1 turn). 665k chars, 1.7k–104k each, skewed large (mean 32k vs 13k overall). ≈3–5 h CPU, once.
- Truth: checkpoint entities (paths, hosts, versions, `backticked` names) that also occur in the stored turns. `turns` holds no tool output, so score against the deterministic note on the same truth, not against 100%.
- Pass = all three: (1) invented entities ≤5% (digest entities found nowhere in the source turns); (2) recall ≥ the deterministic note's + 15 percentage points; (3) blind read: 5 random A/B pairs (digest vs deterministic note), digest preferred in ≥4.
- Thresholds are fixed now; do not move them after seeing results.
- Pipeline rule for any backfill: every digest item carries a verbatim source quote, and items whose quote is not in the source are dropped before PG; long sessions are chunked (map-reduce) so nothing is silently truncated; digests pass §6.4 redaction. Full backfill ≈1.0–1.3M prompt tokens ≈ 16–24 h CPU on `gemma4`.

P4 embedding gate. Postgres FTS only at first. Replay-eval: ~40 queries with known target sessions (recall@5 and MRR as in §6.9; set the threshold after the first run). Add `nomic-embed-text` only if the replay-eval says so, with `keep_alive=-1` (an idle unload would break the p95 <300 ms target) and a ≈250 ms timeout that falls back to FTS.

Service. `lq` and batch jobs share the main `ollama.service` on :11434. Inference runs inside `ollama serve`, so `nice` on a client script does nothing; CPU contention with builds is accepted and `lq` may queue behind a running batch. `/etc/default/ollama` stays untouched.

6.12 P1 as built (accepted 2026-10-09)

§6.2–6.4 and 6.8 stay the design intent. This section records what was built, where and why it differs, how to run it, and what it leaves open. Code: `~/copilot`, a Cargo workspace (edition 2024, `unsafe` forbidden, ≈10k lines including tests, 117 tests; `cargo fmt --all --check` and `cargo clippy --all-targets -- -D warnings` are clean).

Layout and install

- `crates/ctx-core` (library: `redact`, `reader`, `events`, `writer`, `store`, `ingest`, `watch`, `verify`, `report`, `db`, `config`; `sql/views.sql`), `crates/ctx-ingest` (`once | verify | tail`), `crates/ctx-report` (`summary | weekly | baseline | sessions | models | guardrail`), `migrations/0001_core.sql`, `systemd/ctx-ingest.service`, `docs/plan-pdf.html` (the template of `PLAN.pdf`, last bullet).
- Install: `cd ~/copilot && CARGO_PROFILE_RELEASE_LTO=thin cargo install --path crates/ctx-ingest --locked --force`, and the same for `crates/ctx-report` (binaries land in `~/.cargo/bin`). `thin` because the

host's `~/.cargo/config.toml` forces fat LTO and these tools are I/O-bound; a release build takes ≈20 s with sccache.

- Service (user unit, no root): `install -Dm644 systemd/ctx-ingest.service ~/.config/systemd/user/ctx-ingest.service && systemctl --user daemon-reload && systemctl --user enable --now ctx-ingest`. The installed unit is a copy: re-run `install` after editing the repo file; after a code change re-install the binary and `systemctl --user restart ctx-ingest`. Outside a login shell `systemctl --user` needs `XDG_RUNTIME_DIR=/run/user/1000`. Log: `journalctl --user -u ctx-ingest`.
- Linger=no, so the service runs only while you are logged in. Harmless: the CLI cannot write while nobody is logged in, and the first pass after the next login resumes from the stored offsets. `loginctl enable-linger` is optional.
- The unit runs at `Nice=10` with `IOSchedulingClass=idle` (§6.3 "run at nice"), not `SCHED_IDLE`, which would starve the tail for the whole of a long build. `Restart=on-failure`, `RestartSec=15`: exit 1 and exit 3 are both safe to retry (the work is idempotent and the lock clears itself). `KillSignal=SIGTERM`, `TimeoutStopSec=60`: a batch in progress commits first, so every stored offset stays consistent with its rows.
- `PLAN.pdf` is a pandoc + WeasyPrint export of this file (A4; needs `pandoc` 3.1 and `weasyprint` 62). Re-export it after every edit of the plan: `cd ~/copilot && pandoc -f gfm -t html5 --wrap=none --template docs/plan-pdf.html --metadata pagetitle="Copilot workflow optimisation: decided plan" PLAN.md | weasyprint - PLAN.pdf`. Keep `--wrap=none`: pandoc's default wrapping breaks the long page title across two lines inside the CSS string of the page-footer rule, the unbalanced string makes the rest of the stylesheet unparseable, and WeasyPrint then renders an unstyled serif document without any error. A ticked task-list item (§12) shows as a filled box, an open one as an empty box. Last exported 2026-10-09, after P1.

Configuration and exit codes

- PostgreSQL URL: `--database-url`, else `CTX_DATABASE_URL`, else `DATABASE_URL`, else `postgres:///copilot_ctx?host=/var/run/postgresql&user=<$PGUSER|$USER|$LOGNAME>` (without a user sqlx connects as "anonymous" and peer auth refuses it). Copilot dir: `--copilot-dir`, else `CTX_COPILOT_DIR`, else `COPILOT_HOME`, else `~/.copilot`. The unit sets none of them.
- `ctx-ingest` exits 0 ok; 1 error, or the tail lost its lock connection; 2 `verify` failed; 3 another ingester holds the lock. `ctx-report baseline` and `guardrail` exit 2 when a line misses or a proxy stayed worse.
- Logging: sqlx statement logging is off (default filter `info,sqlx::query=error,sqlx::postgres::notice=warn`); an exported `RUST_LOG` replaces it.

Commands

- `ctx-ingest once [--force] [--jobs N]` imports everything new (the first run imports the whole history); a re-run changes nothing. `--force` re-reads every source from the start. It is needed after the redaction rules change: the rules apply when a line is read, stored rows are never re-redacted in place. An interrupted `--force` keeps its last committed chunk; run it again to finish. The "changed rows" it prints are write counts, not distinct rows: a tool call whose start and complete events land in different batches counts twice.
- `ctx-ingest tail` catches up, then keeps the DB current (below). `ctx-ingest verify` runs the acceptance checks (below); it is read-only, prints counts only, takes no lock and is safe while `tail` runs.
- `ctx-report summary | weekly [--weeks 12] | baseline | sessions | models | guardrail`, each with `--as-of <RFC 3339>` (below).

Schema as built (deviations from the §6.2 sketch)

- Tables: `sessions`, `events`, `tool_calls`, `usage`, `turns`, `checkpoints`, `session_files`, `session_refs`, `ingest_state`, `redactions`, `ctx_meta` (key/value; holds `views_sha256`), `_sqlx_migrations`. Keys: events `(session_id, event_id)` (a forked session copies its parent's history, so one event id legitimately appears under several sessions), tool_calls `(session_id, tool_call_id)`, usage `(session_id, src_id)`, turns `(session_id, turn_index)`, checkpoints `(session_id, checkpoint_number)`, session_files `(session_id, file_path)`, session_refs `(session_id, ref_type, ref_value)`.

- Every child table references `sessions(id) ON DELETE CASCADE`: `DELETE FROM sessions WHERE id = …` purges a session completely, and a trigger also forgets its `ingest_state` row (otherwise the ingester would think the deleted session is up to date). The DB is an archive: rows the CLI deletes from its own stores are never deleted here.
- `turns.assistant_response` (the store's own column) instead of `assistant_final`. `turns` and `checkpoints` carry `src_hash` (SHA-256 of the raw source row), so unchanged rows are skipped without re-redacting.
- `events.ctx` is `main` or the sub-agent's `agentId` (1:1 with `parentToolCallId`), so events join `usage` on `(session_id, agent)`. `agent_id` and `parent_tool_call_id` have their own columns; `src_offset` is the byte offset of the line in `events.jsonl`, to fetch the raw event later.
- `ingest_state(source, key, byte_offset, file_size, head_sha, watermark, mtime, updated_at)`: `offset` is a reserved word; `head_sha` (hash of the file's first bytes) catches a replaced file that happens to be larger than the stored offset.
- `redactions(session_id, source, kind, n)`: `source` is `events` (accumulated by the ingester and reset whenever a session is re-read from offset 0, so re-runs never double-count) or `store` (recounted from the mirrored tables).
- `usage` adds `copilot_usage_model`, `request_multiplier`, `duration_ms`, `ttft_ms`, `itl_ms`, `output_ttft_ms` (double precision: the source has fractions), `api_endpoint`, `finish_reason`, `content_filter`; `price jsonb` holds `token_details_json`. `sessions` adds `host_type`, `max_ai_credits`, `summary`, `inserted_at`. `tool_calls` adds `mcp_server`, `result_bytes` (the true size before the cap), `exit_code`, `parent_tool_call_id`.
- Text caps: every string in `events.body`, `tool_calls` and other JSON is cut at 8 KiB after redaction, long-form store columns at 64 KiB. About 7% of tool results are cut; `result_bytes` keeps the real size.
- The views (`v_call`, `v_token_aiu`, `v_turn`, `v_session`, `v_model`, `v_split`, `v_quality`, `v_permission_wait`, `v_weekly`, `v_summary`) live in `crates/ctx-core/sql/views.sql`, not in a migration. At start-up the file's SHA-256 is compared with `ctx_meta.views_sha256` and the views are recreated when it differs (under an advisory lock, so two starts cannot interleave): a view change needs no migration. Never edit `0001_core.sql`: sqlx checksums applied migrations and refuses a modified one (that is why its comment on `ingest_state`, which still says the store watermark covers `assistant_usage_events` only, is left stale; see Ingest).
- Not created by P1: `notes` (P2), `hook_audit` (P3: migration `0002_hook_audit.sql`, created since, §6.13), `hist_chunks` (P4, needs pgvector), `code_chunks` and `worktree_files` (P5).

Ingest

- Sessions = the union of the session directories and the rows of the store's `sessions` (332): 385 on 2026-10-09, of which 359 have an `events.jsonl`. §2's "324 sessions" is the store's row count at the time. Dot-directories (`.session-operation-locks`) are skipped.
- `events.jsonl`: one transaction per batch, with the resume offset written in the same transaction as its rows, so a crash can never leave an offset ahead of its rows. The reader stops at the last complete line (a partial tail waits until it is complete) and ignores NUL padding. Writes are upserts guarded by `IS DISTINCT FROM`, so a re-run changes zero rows.
- Resume plan per file: no state → read from the start; recorded size == size on disk → skip (one `stat`); file shorter than the offset → restart; longer → resume if `head_sha` still matches, else restart. **Known limit:** an in-place overwrite that keeps the file size is not noticed; `once --force` heals it.
- `session-store.db`: opened read-only and read in one deferred transaction, so its tables are mutually consistent. `turns` and `checkpoints` are change-detected by `src_hash`. `assistant_usage_events` is reconciled on every sync against the complete set of source ids already in PostgreSQL (a few ms): a watermark would miss rows after id reuse. Each sync also records the newest row id of six tables (`sessions`, `turns`, `checkpoints`, `session_files`, `session_refs`, `assistant_usage_events`) in `ingest_state` (`source = 'store'`), which `verify` uses. A store row lost from PostgreSQL returns on a plain `once`; a lost event row needs `--force`.
- Idempotency, proven 2026-10-09 on the real data: an incremental `once` added exactly the new lines of the session still being written (+104 events for 104 new lines; the other 358 sessions untouched). `once --force`, run twice, re-read all 359 files (≈215k lines, 0 rejected) and changed only that session's rows (6, then 22 new events); apart from it, count-and-md5 fingerprints of all 9 data tables were identical

before and after, and no table has a duplicate key. The integration test `rerunning_and_forcing_add_no_rows` asserts 0 changed rows on a quiet tree.

Redaction (§6.4 as built)

- Marker `[REDACTED:<kind>:<sha8>]` (first 8 hex digits of the SHA-256 of the secret: equal secrets correlate, nothing is recoverable). Per string, in this order: strip NULs (PostgreSQL text/jsonb reject them), replace base64 blobs of ≥512 chars with `[blob N bytes]`, redact secrets, and only then cap the length, so a secret can never survive as a harmless-looking truncated prefix. Everything stored derives from the redacted text.
- Kinds: the §6.4 list (`pem`, which also catches a block whose END line the CLI's own output limit cut off; `github`; `aws` for AKIA/ASIA/AROA/AIDA; `jwt`; `bearer`; `slack`; `stripe`; `assign`) plus `auth_header`, `url_cred` (`://user:pass@`), `google`, `sk`, `npm`, `gitlab`, `hf`, `pypi`, `sendgrid` and `entropy`. Values of JSON keys that name a secret (`password`, `token`, `apiKey`, `authorization`, …) are redacted whatever they contain. `assign` leaves references alone (`$VAR`, `::` and `/` paths, an existing marker) and stops at a call expression.
- Entropy rule (§6.4's "tune thresholds"): a token of ≥32 chars over `[A-Za-z0-9+_=-]` with Shannon entropy ≥4.3 bits/char and a mix of upper case, lower case and digits. Hex digests and UUIDs stay below the bar (max 4.0 bits). It is skipped for values of identifier keys (suffix `id`, `ids`, `uuid`, `ref`: `toolCallId`, `apiCallId`, …), which are random-looking by design and must stay joinable.
- Dropped before storing: the `data` of `session.binary_asset` (id, mime type and size stay), inline attachment `data`, `binaryResultsForLlm`, `reasoningOpaque` / `encryptedContent`, the tool-result and tool-args copies inside `hook.start` input (they duplicate the `tool.*` events), and everything after the first 512 bytes of a `system.message` (the CLI's own prompt, repeated in every session).
- On the real data (2026-10-09): 9,736 redactions (events 9,690, store 46): entropy 7,921, assign 1,578, url_cred 135, jwt 40, aws 34, auth_header 12, pem 9, bearer 7. The entropy rule makes 81% of them. The contexts sampled (keys and neighbouring words only, never values) were genuine hashes and credential-like tokens (`sha256 = "…"`, client and api ids, nix store hashes). It errs on the safe side (a redacted hash costs a little recall, a missed secret costs more), so it was not tuned. If `history_search` (P4) misses identifiers it should have found, tighten the rule (higher bar, more key exemptions) and run `once --force`.

Tail

- inotify (`notify`) says only *when* to look; *what* to copy is decided by the same code as `once` (resume from the stored offset; an unchanged file costs one `stat`). inotify errs in both directions, hence two rules: watches are always set before a full pass reads anything (a directory that appears between listing and watching raises no event for the files already inside, and the kernel queue can overflow), and a periodic full pass backs the notifications up. Access and metadata events are ignored; otherwise reading `events.jsonl` raises an event that schedules a read, forever.
- Defaults: `--debounce 2s` (files quiet), `--max-delay 10s` (a session that never pauses still lands), `--store-interval 15s` (the store is the costliest source), `--rescan 60s`, `--jobs 4`, pool = jobs + 4. One task does all the work, so two writers never touch the same session. The store may appear after the tail started. A batch in progress commits on SIGTERM or Ctrl-C.
- Measured: caught up in 0.3 s; 14–19 MB RSS; new events of the running session follow with 0 lag.
- Lock: one session-level advisory lock per schema, `pg_try_advisory_lock(hashtext('ctx-ingest:' || current_schema()))`, on a dedicated connection. `once` and `tail` both take it, so a manual `once --force` while the service runs exits 3 ("another ctx-ingest is already running"), and vice versa; `verify` takes none. PostgreSQL drops the lock with its connection. The sqlx pool reconnects silently but the lock connection never does: a kill test showed the tail carrying on without the lock. So the tail checks it (`SELECT 1` every 30 s; hidden option `--lock-check`) and exits 1 when that fails; systemd restarts it 15 s later and it takes the lock again. Verified on the real service (lock backend terminated 23:25:46, watchdog fired 23:26:02, restarted, new lock, lag 0) and by `crates/ctx-ingest/tests/lock.rs` (the real binary; a disabled and a false-positive watchdog each make a test fail). A killed pool connection needs nothing: sqlx logs an INFO `ping on idle connection returned error` and carries on.

Verify (`ctx-ingest verify`; exit 2 on failure)

1. Every `events.jsonl` against `events` and `tool_calls`, up to the byte offset the ingester recorded, so a session still being written cannot cause a false alarm.
2. `session-store.db` against its mirror (keys everywhere; token and price totals for usage), up to the newest row ids the last sync recorded.
3. The secret scan, run by PostgreSQL's own regex engine with patterns written separately from the redactor's, so one mistake cannot hide in both. It is proven first on 17 canaries it must find and on redaction markers it must ignore. It reads decoded JSON leaves, not JSON text (JSON syntax makes `"TokenMs":` and the `\n` after `SECRET=` look like leaks).
4. A fixed point: running the redactor over everything stored must change nothing. This is the only check that covers the entropy rule, which SQL cannot express. Only identifier and hash columns (`session_id`, `event_id`, `ctx`, `src_hash`, `head_sha`, …) are exempt.

- Whatever still disagrees (a session that appeared since the last sync) is looked at again 3 times, 3 s apart; a mismatch that survives is real.
- Result 2026-10-09, also while the tail ran: PASS. Counts match (385 sessions, 214,979 events, 34,285 tool calls, 19,537 usage rows, 1,736 turns, 47 checkpoints, 724 files, 184 refs; usage token totals identical); 17/17 canaries found; 2,246,917 strings in 65 columns, 0 matches; 655,258 values re-read, 0 would change; 0 sessions never read.

Report

- AIU = `nano_aiu / 1e9`; ISO weeks start on Monday; a cold call is exactly §6.8's definition, per `(session, ctx)`.
- The views take their cut-off from the session variable `ctx.as_of`, set by `--as-of <RFC 3339>`. `baseline` defaults to `2026-10-09T11:30:00Z`, the moment §2 was taken, so it keeps reproducing §2 while data keeps arriving (live now: ≈19.6k calls, ≈134k AIU); every other subcommand defaults to now.
- `baseline` compares 8 required lines (exit 2 if one misses) and 38 informational §2 lines (±3%). As of the cut-off:
  - Required, 8/8 pass: AIU 129,366.45 vs ≈129.4k (−0.03%, tolerance ±1%); calls 18,722 vs ≈18.7k; sessions with usage 188; token split cache_read 53.03 / cache_write 29.91 / output 14.72 / input 2.34 % (±0.1 pt), and the split sums to the total AIU.
  - Informational, 35 of 38 within tolerance. The 3 that are not: `sessions_gt_2000_aiu` 14 vs 15 and `aiu_pct_sessions_gt_2000` 53.40% vs 55% (a session crossed 2,000 AIU after the cut-off and the DB shows 15 now, so §2's figure was taken a little later than 11:30Z); `permission_requests_distinct` 6,907 vs 6,607.
  - Permission prompts: there are 7,195 `permission.requested` events and 6,907 distinct `requestId`s (288 ids repeat across forked sessions that copy their parent's history). 6,607 looks like a transposition of 6,907 (a guess). The wait statistics match (86.10 h vs 86; ≥60 s 537 vs 540; ≥300 s 129 vs 129), so the events themselves are right. **P3's replay set is the 7,195 events (or the 6,907 distinct), not 6,607.**
- `weekly` shows the latest `--weeks 12` rows (spend, per-turn ratios, cold and autopilot share, quality proxies); `--weeks 40` shows everything. A week with neither events nor usage has no row: the DB has none for the ISO weeks starting 2026-08-03, 08-10 and 08-17. `usage` starts 2026-07-24 (events go back to 2026-05-26), so the earlier weeks show messages and proxies but no AIU.
- `guardrail` follows §6.8 with four per-100 proxies: aborts per 100 user messages (baseline 2.85), failures + retries + errors per 100 assistant turns (1.44), `session.task_complete` per 100 user messages (10.48; lower is worse), permission waits ≥300 s per 100 user messages (4.97). Weeks with fewer than 30 user messages are ignored (`--min-messages`); it prints the table and exits 2 when a proxy was more than 25% worse (`--threshold`) for 2 consecutive weeks. `--since` defaults to `2026-10-12`, the Monday after the §2 cut-off, so it stays quiet until post-baseline weeks exist.
  - Snapshot 2026-10-09: with `--since 2026-09-28` the permission-wait proxy is already worse for two weeks (7.08 and 16.38 per 100 messages; the second week is not over), before any lever was applied.

- Not implemented: the correction-phrase proxy of §6.8 (its regex list is still TBD).

Measured

- 2026-10-09: backfill of 359 files (≈215k lines) 5.1 s; forced re-read ≈3.5 s; incremental pass ≈0.3 s. DB 703 MB (events 266 MB, tool_calls 54 MB, usage 20 MB).

Tests

- `CTX_REQUIRE_DB=1 cargo test --workspace`: 117 tests (91 unit; 22 integration in `ctx-core/tests` for ingest, report, tail and verify; 2 unit and 2 lock tests in `ctx-ingest`). Without a reachable database the DB tests print SKIPPED and pass, unless `CTX_REQUIRE_DB` is set; `CTX_TEST_DATABASE_URL` overrides the URL. Each test works in its own schema `ctxtest_<name>_<pid>` (the pool pins `search_path`) and drops it; at most 4 tests use PostgreSQL at once. A killed run can leave `ctxtest_*` schemas: list them with `psql -d copilot_ctx -c "select schema_name from information_schema.schemata where schema_name like 'ctxtest%'"`, then `DROP SCHEMA … CASCADE`. The tests never touch `public`, `~/.copilot` or the running service (the lock tests spawn the real binary on an empty temp dir).

Pins and gotchas for later phases

- `rusqlite` is pinned to 0.39: 0.40 needs `libsqlite3-sys` 0.38, while sqlx 0.9's optional sqlite driver (disabled here but still resolved into the lockfile) accepts only older ones, and two crates with `links = "sqlite3"` cannot coexist. Revisit when sqlx moves.
- sqlx 0.9: `query*()` accept only `&'static str` or `AssertSqlSafe(String)`; use `raw_sql` for multi-statement SQL; there is no numeric decoder, so cast sums to `bigint` or `float8`.
- Later phases read the redacted tables, never `events.jsonl` (D5): P4's `hist_chunks` is built from `turns`, `tool_calls`, `checkpoints` and `events`.

Open (nothing here blocks P3)

- You (sudo): pgvector, and `listen_addresses = ''` (§6.1).
- `~/copilot` is a git repo with no commits and the placeholder identity `Your Name <you@example.com>`; nothing was committed.
- Known limits: a same-size in-place overwrite (Ingest); the `store` rows of `redactions` do not count what was redacted inside `sessions.summary` (the text itself is redacted, scanned and fixed-point checked); the correction-phrase proxy.

6.13 P3 as built (installed in shadow mode 2026-10-10; not enabled)

§6.6 stays the design intent. This section records what was built, what it measured on the real history, what the live CLI turned out to do, and what is left for you. Code: `crates/ctx-hook` in `~/copilot` (≈11.6k lines of source with their unit tests plus ≈5k lines of integration tests, 200 tests; the whole workspace passes 319 under `CTX_REQUIRE_DB=1 cargo test --workspace`; `cargo fmt --all --check` and `cargo clippy --workspace --all-targets -- -D warnings` are clean). Nothing is committed.

What you do next (the hook is installed but only logs)

- Open a NEW Copilot session: hooks are read when a session starts, so sessions that are already running (this one included) have no hook. Use the CLI as usual for 3 days.
- Then run `ctx-hook review`. It imports the spool, joins the audit rows with what the CLI logged about the same requests, and prints four gates (three days of log under the policy now in force, no one refused a request the hook would have allowed, p95 <50 ms, the hook's directory is the real one) and a verdict line. It changes nothing and always exits 0. It is empty until the first real row: that row starts the 3-day clock.
- To enforce, set `mode = "enforce"` in `~/.config/ctx/permissions.toml`. It takes effect on the next request, even in sessions that are already running (every hook process reads the file). `mode = "shadow"` goes back; `mode = "off"` makes the hook exit at once and log nothing. `~/.copilot/hooks/ctx.json` needs no edit for any of this.
- Widening the allow-list or the roots is your call (numbers under Results). Roots and deny patterns go in the same file (`trusted_roots`, `deny`); they can only add to the built-in ones. A broken file (unreadable,

unknown key, bad value) makes the hook pass everything and never enforce (policy id `v1:config-error`, shown by `explain` and `review`).

Installed

- `~/.cargo/bin/ctx-hook`. `ctx-ingest` and `ctx-report` were reinstalled too: both run the migrations at start-up, and the old binaries refuse a database that holds a migration they do not know. Migration `0002_hook_audit.sql` was applied by restarting `ctx-ingest.service`.
- `~/.config/ctx/permissions.toml` (`mode = "shadow"`, comments explain every key and the built-in lists) and `~/.copilot/hooks/ctx.json` (`permissionRequest` → `ctx-hook permission`, `timeoutSec` 5).
- Re-install after a code change: `cd ~/copilot && CARGO_PROFILE_RELEASE_LTO=thin cargo install --path crates/ctx-hook --locked --force` (one crate ≈25 s). After a new migration also reinstall `ctx-ingest` and `ctx-report`, then `systemctl --user restart ctx-ingest`. `CARGO_TARGET_DIR=$PWD/target/install` keeps the install build apart from the test build.

Commands

- `ctx-hook permission` is the hook. `explain [--cwd DIR] <command>` prints mode, policy id, config error if any, verdict and roots (exit 0 = would allow, 1 = would pass).
  `replay [--since D] [--limit N] [--top N]` runs the policy over every shell request the CLI ever asked a person about. `review [--since D] [--top N]` is above. `bench [--runs 200] [--warmup 10]` runs this binary as a hook the way the CLI does, with the config and database settings of its environment, and deletes its rows afterwards (session ids `ctx-hook-bench-<pid>`, which `review` ignores). Point `CTX_DATABASE_URL` at a scratch schema (`…&options=-c%20search_path%3D<schema>`) so the real table does not use up ids.
- Modules: `shell` (tree-sitter-bash 0.25 analyzer), `cmds` and `opts` (allow-list with per-command flags), `git`, `trust` (roots, deny list), `vars`, `policy`, `config`, `payload`, `audit`, `hook`; around them `oracle` (an independent, deliberately over-suspicious lexer that distrusts the analyzer), `replay`, `review`, `bench`.

The hook (what it does and never does)

- `permission` is dispatched before the argument parser (its usage errors exit 2), works on a thread with a 64 MiB stack, gives up silently after 2 s, and always exits 0. Exit 2 means DENY, so the hook has no deny verdict at all: its only output is `{"behavior":"allow"}`, every other outcome (pass, bad payload, config error, panic, timeout) is empty output, which is the normal prompt.
- Order: judge, write the audit row, and only then, in enforce mode, print the allow. The row goes to PostgreSQL if it answers within 40 ms; otherwise it is appended to the spool `$XDG_STATE_HOME/ctx/hook-spool.jsonl` (default `~/.local/state/ctx/`, 8 MiB cap, imported by `review`, each row once through the unique `uid`). An allow is never sent for a request whose row reached neither place. A full spool means nothing is allowed until `review` has drained it.
- A line is allowed only when ALL hold: the analyzer understood every part of it (accepted syntax is a short whitelist: simple commands, pipelines, `&&`/`||` lists, plain and quoted words, comments, redirects; any other node kind passes as `node:<kind>`, and so does text the parse tree does not account for, `gap`); every command is on the allow-list with only listed flags (`cd grep rg cat ls find head tail wc sort uniq jq echo printf which stat file du readlink`; `git` with `log show diff status branch rev-parse ls-files blame describe`); every path it can touch lies inside a trusted root and outside the deny list, judged as the kernel resolves it (symlinks, `..`, wildcards expanded against the disk; the one exception is a link named in a git command, see Findings); redirects go only to `/dev/null` or duplicate a descriptor; and the environment cannot change how a command parses (`POSIXLY_CORRECT`, `CDPATH`, `GIT_*` that redirect git, other `GIT_CONFIG*` than the three the CLI itself exports, ripgrep's config file).
- Built-in roots `~/kms*` and `~/pr-*`. Built-in deny list: `.env*`, `*.pem`, `*.key`, `*.p12`, `*.pfx`, `id_rsa*`/`id_dsa*`/`id_ecdsa*`/`id_ed25519*`, `.netrc`, `.npmrc`, `.pypirc`, `.git-credentials`, `.pgpass`, `~/.ssh/**`, `~/.gnupg/**`, `~/.aws/**`, `~/.kube/**`, `~/.config/gh/**`, `~/.docker/config.json`, `~/.copilot/config.json`.
- `hook_audit` (14 columns): `id, uid, ts, session_id, tool, cmd_sha, cmd_preview` (500 bytes), `cwd, decision` (`allow` | `pass`), `rule` (`allow:<commands>` or `pass:<reason>[:<command>]`), `shadow, latency_us, policy, inserted_at`. No foreign key to `sessions` (the hook fires before the ingester has seen a new session), so deleting a session does not purge its audit rows: delete them by `session_id`. Text is

redacted like everything else; `cmd_sha` is the SHA-256 of the redacted text and equals the hash of the same command in `events`, which is how `review` joins a row to the CLI's own question (verified live). `ctx-ingest verify` scans the table too (2026-10-10, with rows present: PASS).

- The policy id is `v<POLICY_VERSION>:<8 hex digits of a hash of the roots and the deny list>`; the mode is not part of it. The 3-day gate of `review` counts only the shadow rows written under the policy now in force (a log made under another policy says nothing about this one), so changing the roots or the deny list restarts the 3 days on purpose.

Results (§5 acceptance: p95 <50 ms met, replay with 0 false allows met; the 3 days of shadow log have not started, they begin with the first real row)

- Latency (installed release binary, thin LTO, 200 runs after 10 warm-up, scratch schema): process start to exit p50 9.6 ms, p95 10.5 ms, p99 10.9 ms, max 11.4 ms against the budget of 50 ms, including the PostgreSQL insert; inside the process p50 5.6 ms, p95 5.9 ms. In the CLI's own log `hook.start` → `hook.end` takes 10–12 ms.
- Replay (`ctx-hook replay`, policy `v1:78077c33`, home `/home/cscm`): every `permission.requested` event of kind `shell` (the only kind this hook is asked about; one per session and request id): 4,509 of them, 2026-06-16 .. 2026-10-09, 3,995 distinct command/directory pairs; 4,242 asked about the command and 267 about a path. Each is judged with the disk as it is now (1,350 ran in a directory that no longer exists and are judged conservatively, so the numbers are a floor).
  - **Allowed 423 of 4,509 (9.4%)**, 389 distinct commands; 404 of the 3,159 whose directory still exists (12.8%). By prompt kind: 423 of 4,242 command prompts, 0 of 267 path prompts. Rules: `allow:grep+head` 128, `allow:grep` 43, `allow:cd+git-diff+head` 23, `allow:cat` 21, `allow:git-diff` 15, `allow:cd+grep+head` 13, `allow:git-status` 12, `allow:ls` 12.
  - **0 false allows.** It would have allowed 0 of the 34 requests a person denied, 0 of the 7 cancelled, 0 of the 11 denied unattended. The oracle flags 0 of the 389 allowed commands, and 250,000 generated lines (fixed seed) gave 0 flags. The CLI itself marked 2 allowed commands as not read-only (`git log … --grep="a\|b"…` and a `git show --stat --format=…`): its conservatism, not a hole.
  - **Waiting, the finding:** 4,498 answered requests waited 80.5 h in total (29.6 h with each wait capped at 5 min). The allowed ones waited 4.6 min in all (0.1%; median 0 s, p90 0 s, longest 1.8 min): they are the requests the CLI approves instantly anyway. Under 1 s: 2,276 requests (418 allowed, 3.0 min). 1–5 s: 760 (1 allowed). 5–30 s: 716 (2). 30 s–5 min: 630 (2; 16.7 h, 56.4% of the capped time). 5 min or more: 116 (0 allowed; 9.7 h, 32.6%). **Of the 1,462 requests that waited 5 s or more, the policy covers 4.** The §6.6 allow-list therefore removes almost none of the waiting, and none of the ≥5 min waits that make the cache go cold (§10).
  - What stops the rest (first reason): `cmd-not-allowed` 1,190 (10.5 h waited), `node` 962 (10.2 h), `path-untrusted` 1,280 (4.8 h), `redirect-target` 155, `git-subcommand` 106, `gap` 95, `name-node` 93, `meta` 47, `cd-with-or` 36, and a tail. The replay alone also has `stored-truncated` 47 and `stored-redacted` 20: the stored text was cut or redacted, so the command cannot be judged.
  - **What if (an upper bound; extensions are your decision).** 905 requests (20.1%) fail only because a command is not on the list. Adding commands one after another (any arguments, every other rule as it is) lifts the allowed share to sed 11.6%, cargo 12.7%, docker 13.8%, curl 14.7%, python3 15.7%, rm 16.4%, xargs 17.0%, gh 17.7% … 22.7% after 25 additions. The wait each removes is small (sed 9.8 min, cargo 37.1, curl 32.4, rm 22.0, python3 19.6, out of 80.5 h), and most of those commands are not safe to approve unseen (`rm`, `sudo`, `curl`, `docker`, `bash`, `python3`, `xargs`). Roots: 1,280 requests stopped first on a path outside the trusted roots. The report tries 20 directories, each ALONE with every other rule as it is, and counts the requests that one more root would have allowed: 150 in all (3.3% of all requests), 6.7 min of waiting. Per directory: `~/ardupilot` 32, `~/ardupilot_gazebo` 22, `~/gz` 16, `~/keycloak.worktrees` 12, `~/t16` 10, `~/jd2` 9, `~/password-store` 9, `~/keycloak` 8, `~/linux-socfpga` 6, one `~/pkcs…` directory 6, `~/wine` 5 (but 6.3 min of the 6.7 min), nine others 1–3 each. A root vouches for everything under it that the deny list does not name: keep directories with secrets out.
- The early estimate of 2.6% (113 of 4,281 distinct requests "solely allow-listed") was a crude name check and too low; the replay's 9.4% supersedes it.

What the live CLI does (1.0.95, four nested `copilot -p` runs, ≈7.9 AIU in all)

- Payload as captured (`payload::CAPTURED`, parsed by the tests): `{"hookName":"permissionRequest","sessionId","timestamp" (ms),"cwd","toolName":"bash","toolInput":{"command"},"permissionSuggestions":[]}`; `requestSandboxBypass` appears in `toolInput` only when true.
- The hook fires for EVERY shell request, including the ones the CLI would approve by itself and never asks a person about. Those rows show as "never asked" in `review`, and the hook's decision is moot for them. Per the hooks reference `read` and `hook` requests are decided before hooks run, so reads never reach this hook (D8's "reads in sibling worktrees" cannot be done here; every live row was a `bash` request, none came from the `view` tool).
- An enforced allow: `hook.end` carries `{"behavior":"allow"}`, the CLI still logs `permission.requested` and then `permission.completed` with `result.kind = "approved"` and no `decisionSource`, and the command runs. A pass leaves the normal flow alone: an unanswerable request ends `denied-no-approval-rule-and-could-not-request-from-user` with `decisionSource: "unattended_fallback"`.
- `cd` does not persist between shell calls: in the history, `commandSegments[].workingDirectory` equals the session's directory in 925 of 925 cases. So the payload's `cwd` is the directory the command really runs in. `review` still compares it with the CLI's `workingDirectory` and with the session's directory (live runs: 0 differences), because a reused async shell keeps the directory it started in.
- Hook definitions load at session start (user level `~/.copilot/hooks/*.json`, or `$COPILOT_HOME/hooks`). A repo-level `.github/hooks/*.json` in an untrusted scratch folder was NOT loaded (the nested run's log, at info level, says nothing about why). The hook's `permissions.toml` is read on every request.
- The CLI exports `GIT_CONFIG_COUNT=3` (`safe.bareRepository=explicit`, `credential.interactive=never`, `core.fsmonitor=false`) to the shells it starts (observed on 1.0.95); the policy accepts exactly these three and refuses any other `GIT_CONFIG*`. Whether the hook process gets the same environment as the shell was not checked.
- The CLI rewrites `~/.copilot/config.json` at every process start, nested runs included (its mtime is the start of the newest `copilot` process). After the nested runs it was checked: same top-level keys, `trustedFolders` still only `/home/cscm`. Nothing here edits it, and the deny list names it.

Testing enforcement without enabling it (done 2026-10-10, 0.9 AIU; repeat it before you flip the real switch if you want a last end-to-end proof)

- `mkdir -p /tmp/p3home/hooks /tmp/p3cfg/ctx /tmp/p3live`. `/tmp/p3cfg/ctx/permissions.toml`: `mode = "enforce", trusted_roots = ["/usr/lib/postgresql"]`. `/tmp/p3home/hooks/ctx-enforce.json`: the hook of `ctx.json` plus `"env": {"XDG_CONFIG_HOME": "/tmp/p3cfg"}`. `ln -s ~/.copilot/config.json /tmp/p3home/config.json` gives the scratch home the login; the CLI replaces the link with a copy of the file at start, so the scratch home then holds your credentials: delete it afterwards.
- `cd /tmp/p3live && COPILOT_HOME=/tmp/p3home env -u COPILOT_AGENT_SESSION_ID -u COPILOT_LOADER_PID copilot -p '<run, one per tool call, never retrying a refusal: ls /usr/lib/postgresql; cat /etc/hostname; touch /tmp/p3live/should-not-exist>' --model gpt-5-mini --max-ai-credits 30 -s --no-ask-user --no-auto-update --no-custom-instructions --disable-builtin-mcps`.
- Result (2026-10-10): `ls /usr/lib/postgresql` ran without a prompt (rule `allow:ls`); `cat /etc/hostname` (`pass:path-untrusted`) and `touch /tmp/p3live/should-not-exist` (`pass:cmd-not-allowed:touch`) were denied unattended, and the file was not created. The three rows carry `shadow = f`, policy `v1:43d8dc5d` and 4.7–5.9 ms. `COPILOT_HOME` replaces `~/.copilot`, `hooks/` included, so the real shadow hook did not fire in that run, and the hook's `env` points it at the scratch config, so the real `permissions.toml` was not involved. The run's events stay in `/tmp/p3home/session-state/<id>/events.jsonl` (the ingester does not read them), but its hook rows go to the real `hook_audit`: delete them (`DELETE FROM hook_audit WHERE session_id LIKE '<first 8 characters of the session id>%'`) and `rm -rf /tmp/p3home /tmp/p3cfg /tmp/p3live`. The same goes for any nested `copilot -p` run made while the real hook is installed: its shadow rows are real rows and would start the 3-day clock early.

Findings worth keeping

- The analyzer is a whitelist, and the tests were written to break it. `tests/mutation.rs` splices dangerous fragments (operators, expansions, redirects, wrappers such as `sudo`, `eval`, `xargs`, `bash -c`, odd

whitespace and control characters) into every position of every allowed base command, and none of the results may be allowed; it found that invisible characters (zero-width, bidi, control) must be refused (`invisible-char`, `control-char`). An unquoted backslash and `{}` are refused, so `find -exec … \;` never reaches the validator. tree-sitter-bash attaches a trailing redirect to the whole list or pipeline (handled) and drops a lone `-` before a redirect (the `gap` check refuses it). Setting the policy against the independent oracle (`tests/differential.rs`: 250,000 generated lines; and the replay) hardened the oracle as well.

- A link named in a git command is allowed, because git shows where a work-tree link points (the link's text) and never what it leads to. The policy depends on that: it resolves links in the operands of every other tool but not in git's. `tests/differential.rs` runs the real git (2.47.3) in a repository whose links lead to canaries, over 76 commands the policy allows, and looks for the canaries in the output. Other git edges that shaped the rules: `git diff A B` with two paths outside the work tree silently becomes `--no-index` and reads arbitrary files (so every word that could be a path must be relative, without `..`, and `--no-index` is refused), and the repository's own config can run programs (`core.fsmonitor`, `diff.<driver>.textconv`), which is why the work tree must lie inside a trusted root.
- The CLI's `commands[].identifier` is often the whole line, not a command name, so a per-command comparison with the CLI's `readOnly` flags is meaningless; the replay compares per request (the 2 disagreements above).
- Two bugs came from tests, not from reading: a `tokio::time::timeout` built outside a runtime in the DB path, and a spool `drain` that classified a database error wrongly.
- For P2: `ctx-hook session-start` belongs in this crate. Copy the shape of `hook::permission` (dispatch before clap, work on a big-stack thread, watchdog, exit 0 with empty stdout on any error), the 40 ms database deadline with a spool for rows that must not be lost, and `bench` for the latency gate (P2's budget is p95 <100 ms; the whole of this hook takes ≈10 ms).

Residual risks (none is a reason not to run in shadow mode; read them before `enforce`)

- An allow from the hook ends the decision (hooks reference): the CLI's own rules, session approvals and any deny rule are not consulted for it. The CLI config has no deny rules today (`permissions-config.json` holds only `tool_approvals`); re-check before adding one.
- Recursive content tools (`grep -r`, `rg --hidden`, `find`, `git diff`, `git log -p`) can print deny-listed files that live under a searched trusted root, or tracked files the deny list does not name: the deny list is matched against named paths and expanded wildcards. A broad root widens this; the deny list is the safety net.
- A trusted repository steers its own commands (its `.git/config`, hooks, aliases); environment variables that do so are refused, repository files are trusted as the repository is. Earlier user-approved commands could have changed the environment or config that an allow-listed command then reads, and the policy reads the hook process's environment as a stand-in for the shell's, which is not necessarily the same.
- Unknown: whether the persistent shell is interactive (history expansion) and whether `dotglob`, `globstar` or `GLOBIGNORE` are set in it.
- The replay judges against today's disk, and its roots what-if tries directories one at a time. A hook timestamp more than 10 minutes off the clock is replaced by the clock, so such a row cannot be joined to its `hook.start`.

Open

- The 3 days of shadow log, and the decision to enforce: yours. So are the extensions (commands, roots): the replay shows a wider list would answer more requests but still little of the waiting.
- The `copilot` alias has no `--add-dir`. Adding `--add-dir ~/kms…` would make the CLI itself trust those directories for reads and path prompts; not applied, since it widens what the CLI approves unasked.
- Still yours from P1: pgvector, `listen_addresses = ''`, linger.

7. Verified CLI facts (v1.0.94; P3's facts were checked live on v1.0.95)

- Hooks: `~/.copilot/hooks/*.json` or inline in `~/.copilot/settings.json`. Command hooks take `exec` + `args` (no shell) or `bash`; default `timeoutSec` 30; command-hook timeouts fail open; `preToolUse` command hooks fail closed (not used).
- `sessionStart` input `{sessionId, timestamp, cwd, source: startup|resume|new, initialPrompt?}`, output `{additionalContext}`; several hooks' contributions are joined with a blank line.
- `permissionRequest` output `{behavior: allow|deny, message?, interrupt?}`; empty output falls through; exit 2 = deny; `allow` never pre-approves a sandbox-bypass request. Optional `matcher` regex on `toolName`. Checked live on 1.0.95 for P3 (§6.13): input `{hookName, sessionId, timestamp (ms), cwd, toolName, toolInput: {command, requestSandboxBypass only when true}, permissionSuggestions}`; the hook fires for every shell request, including those the CLI approves by itself; per the docs it never fires for `read` requests (not exercised with the `view` tool); an enforced allow shows as `permission.requested`, then `permission.completed` with `result.kind: approved` and no `decisionSource`; per the docs an `allow` ends the CLI's decision (its deny rules would not be consulted; not tested); the CLI exports `GIT_CONFIG_COUNT=3` to its shells; repo-level `.github/hooks/*.json` was not loaded in an untrusted scratch folder; `COPILOT_HOME` replaces `~/.copilot` including `hooks/`; the CLI rewrites `~/.copilot/config.json` at every start.
- HTTP hooks to localhost need `COPILOT_HOOK_ALLOW_LOCALHOST=1` (not used). If the session sandbox is ever enabled, hooks run inside it: grant the PG socket and config paths via `sandbox.userPolicy`.
- `--context <default|long_context>` overrides the persisted `contextTier`. Also `--reasoning-effort`, `--max-ai-credits <n>`, `--autopilot`, `--max-autopilot-continues` (default 5). `SessionLimitsConfig` has only `maxAiCredits`; hitting it pauses for a decision. It cannot be persisted (verified 2026-10-09): `copilot config` rejects every limits key and no env var exists, so it comes only from `--max-ai-credits <n>` at launch (min 30) or `/limits set max-ai-credits <n>` in session (survives `/clear` and `/new`). The unit is AIU (`total_nano_aiu/1e9`; 1 AI credit = $0.01). Usage is known only after a call returns, so one call can overshoot.
- Compaction triggers at ~80% of `tokenLimit`: 200k → ~160k (10 observed), 936k → ~750k (2 observed). 94 `session.compaction_start` events overall; 79 older ones carry no limit.
- `/subagents` sets default and per-agent models. Agents emitting `subagentStart/Stop`: explore, task, code-review, rubber-duck, research, security-review (not general-purpose).
- Built-ins that stay: `session_store_sql`, `/chronicle`, `/memory`, `/search`, `/context`, `/usage`, `/limits`.

8. Environment

- Host `t16`: Debian 13 (trixie), i7-1265U (12 threads), 31 GB RAM, no NVIDIA GPU (Iris Xe), `/home` 483 GB free. systemd `--user` running, Linger=no (`ctx-ingest.service` lives there, §6.12).
- Copilot CLI 1.0.95 (`~/.cargo/bin/copilot`; it was 1.0.94 when this plan was written). `~/.copilot/settings.json`: model `claude-opus-4.6` (looks stale: sessions have run sonnet-5 since 2026-08-29, check `/model`), `effortLevel: max`, `contextTier: long_context`.
- Ollama 0.31.2 on 127.0.0.1:11434 (system unit `ollama.service`, runs as `cscm`, CPU-only: `size_vram` 0) with `llama2-uncensored:7b`, `llama3.2:3b`, `qwen2.5-coder`, `gemma4` (no embedding model yet). `http://localhost:11434` is already in `allowedUrls`. Speeds in §6.11.
- Installed: uv, cargo, node, go, rg, ctags, sqlite3, jq, podman 5.4, docker 26, puppet (apply only), graphify 0.9.11. Not installed: pgvector, any MCP server, any repo-level hook (none in kms, artifactkeeper, wine, step, gog when this was written). The only user hook is P3's `~/.copilot/hooks/ctx.json` (`permissionRequest`, shadow mode, §6.13); P2 adds `sessionStart` to it. PostgreSQL 17.11 has been installed since (§6.1).
- `sudo` needs a password. `~/copilot` is the project home: the Cargo workspace of §6.12 plus this file (it was empty when this plan was written).
- `~/kms/AGENTS.md` is 32 KB (≈8k tokens; `CLAUDE.md` is a symlink) and team-owned: not touched.

9. Risks

Risk	Mitigation
Compaction at ~160k drops detail	Handoff note + `history_search`; watch correction-phrase rate; `copilot-long` for opt-in
Permission hook is a security surface	Shadow mode, tiny allow-list, tree-sitter-bash, never exit 2, audit table
A hook `allow` ends the CLI's own permission checks (its deny rules, earlier approvals); recursive tools can print deny-listed files under a trusted root	No CLI deny rules exist today; re-check before adding one; keep roots narrow; residual risks in §6.13
Secrets in PG or embeddings	Redact at ingest, count-only tests, socket-only PG, peer auth
Org content-exclusion policy (patterns unknown locally)	Deny-list in indexer and recall; default-exclude secret-like files; confirm with the admin if in doubt
Hook latency or failure	`exec` binary, 5 s timeout, always exit 0 with empty stdout on error
MCP schema tax (~450 tokens/tool/call)	≤3 tools, terse descriptions, measure `tool_tokens`
Rust costs ~3× Python in effort	P1 and P3 first, P5 gated
Estimates are upper bounds and overlap	Weekly report decides; expect −30–40%
Ollama CPU contention with builds; `lq` queuing behind a batch job	Accepted (2026-10-09). Inference runs inside `ollama serve`, so `nice`/`ionice` on a client script has no effect; batch work uses the main `ollama.service`
Local-model digests invent decisions or carry secrets	Verbatim-quote rule, §6.4 redaction before PG, bake-off gate (§6.11)
An `lq` call blocks past the 300 s cache TTL	Input capped at ~6k chars, 180 s hang guard
300 s cache TTL may differ per model	Read `modelCacheState.cacheTtlSeconds` in `session.usage_checkpoint`

10. Savings estimate (share of baseline AIU, non-additive)

Lever	Estimate	Basis
Default tier, compaction at ~160k	15–20%	Cache reads above 150–200k (upper bound)
Fresh start + handoff vs stale resume	up to 8–10% (overlaps)	247 user-initiated cold calls = 12.4%
Permission hook	≈0% as specified (§6.6); unproven otherwise	The replay (§6.13) allows 9.4% of shell requests but 0.1% of the waiting and 0 of the 116 waits ≥5 min, which are the ones that rebuild the cache. Earlier basis, not supported: ~55 approval-wait rebuilds ≈2.1% plus stalls
Sub-agent routing	3–4%	7% on `gpt-5.4` at 4.1 AIU/call vs 0.8 for mini
Batching nudge	3–5%	≤9.5% of calls mergeable
Poll cadence ≤240 s	~1%	27 cold rebuilds
Delegating noisy loops	1–2%	builds 2.9% + `read_bash` 0.8% carry
`code_search`	1–3%	Exploration carry ~12–17%, lexical only
Ollama `lq` (P6)	<1% (≈0.2%)	274 one-call turns = 2.7% of AIU; absorbing a quarter at ≈3.8 AIU saves ≈30 AIU/week of ≈13k
Soft caps, autopilot alias	not quantified	Decision points on the sessions that make up 72% / 55% of spend

11. Optional levers if the gap persists (decide from the weekly report)

- Effort `max` → `high` outside `~/kms*` (≤2%, quality risk).
- Trial `/model auto` (unquantified; 195 `session.auto_mode_resolved` events exist to study).
- Disable the 22 never-invoked skills (≈1.2k tokens, ~0.4%): actix-web-middleware-audit, caveman, ci-efficiency, ci-fix, cryptography-review, git-guardrails-claude-code, handoff, improve-codebase-architecture, kms-caveman, kms-release-notes, meta-security, mise, playwright-kms, pre-release, rust-async-refactor, rust-error-propagation, rust-panic-audit, rust-patterns, rust-refactor, rust-review-all, rust-simplify, simplify-code.
- Start long interviews/planning in a fresh session (`ask_user` waits ≈2%).
- `--max-autopilot-continues 3`.
- Fix the stale `model` default in `settings.json` if new sessions really start on opus-4.6.

12. Status

- ☐ P0a: `contextTier: default` (done 2026-10-09; confirm via the next new session's `session.start`), `copilot-long` alias (done 2026-10-09, `~/.zshrc`), soft cap (done 2026-10-09: `copilot` alias with `--max-ai-credits 1000` in `~/.zshrc`; confirm `/limits` in the next new session)
- ☒ PostgreSQL 17 + `pg_trgm`, DB `copilot_ctx` (done; peer-auth `select 1` confirmed 2026-10-09)
- ☐ `postgresql-17-pgvector` + `CREATE EXTENSION vector` (you, sudo; needed before P4), and optionally `listen_addresses = ''` (§6.1)
- ☒ P1: ingester + report, baseline reproduced (done 2026-10-09, §6.12: row counts match the sources, re-runs add 0 rows, report reproduces §2 (129,366 AIU vs ≈129.4k, token split within 0.1 pt), 0 secret-pattern matches in 2.25M scanned strings; `ctx-ingest.service` keeps the DB current)
- ☐ P3: permission hook, shadow → enabled (built and installed in shadow mode 2026-10-10, §6.13: p95 10.5 ms, replay 0 false allows; open: 3 days of shadow log from your next new session, then `ctx-hook review` and your decision on `mode = "enforce"`)
- ☐ P2: handoff note + `sessionStart` hook, deterministic digests for all old sessions
- ☐ P2 bake-off (§6.11): `gemma4` digests vs deterministic note on the 21 checkpointed sessions; Ollama backfill only if all three gates pass
- ☐ P0b: sub-agent routing, nudges, autopilot alias
- ☐ P4: MCP history tools (Postgres FTS); replay-eval decides `nomic-embed-text` (§6.11)
- ☐ P5: `code_search` (gated)
- ☐ P6: `lq` trial; rule check after 14 days (≥20 calls, ≥70% useful, else delete)
- ☐ Weekly report review; decide §11 levers